IntelliDoc

Comprehensive Project Documentation

Gaurav Jangir

October 10, 2026

Abstract

This documentation provides a comprehensive overview of the IntelliDoc project. It is divided into two main chapters: an in-depth breakdown of the project's architecture, implementation, and results; and a glossary of technical terms explaining the underlying artificial intelligence and software engineering concepts used in the system.

Contents

1 About the Project **2**
- 1.1 The Idea . . . 2
- 1.2 Planning & Requirements . . . 2
- 1.3 Architecture . . . 2
- 1.4 Implementation Details . . . 3
  - 1.4.1 Document Ingestion Pipeline . . . 3
  - 1.4.2 Embedding and Indexing . . . 3
  - 1.4.3 Retrieval-Augmented Generation (RAG) . . . 3
- 1.5 Advantages & Key Features . . . 3
- 1.6 Results and Outputs . . . 4

2 Project Terms & Concepts **5**
- 2.1 What is RAG (Retrieval-Augmented Generation)? . . . 5
- 2.2 What are Vector Embeddings? . . . 5
- 2.3 What is FAISS? . . . 5
- 2.4 What is a Large Language Model (LLM)? . . . 5
- 2.5 What is Chunking? . . . 6
- 2.6 What is FastAPI and Streamlit? . . . 6

Chapter 1

About the Project

1.1 The Idea

With the exponential growth of digital documents, extracting specific, accurate information from lengthy PDFs has become a time-consuming challenge. Traditional keyword search is often insufficient because it lacks semantic understanding. The idea behind IntelliDoc was to build an "Innovation Catalyst" tool—a system that reads, understands, and retrieves information from documents intelligently, acting as a personal research assistant.

1.2 Planning & Requirements

The project was designed with a focus on modularity, speed, and accuracy. The core requirements were:

- **High-Fidelity Ingestion:** Ability to parse PDFs, including complex structures like tables, without losing semantic context.
- **Low-Latency Retrieval:** Fast vector similarity search to find relevant information instantly.
- **Factuality & Grounding:** The LLM must not hallucinate; every answer must be strictly derived from the uploaded documents.
- **User Experience:** A clean, intuitive web interface for uploading documents and conducting chat-based Q&A.

1.3 Architecture

IntelliDoc follows a modern, decoupled client-server architecture:

- **Frontend (Streamlit):** Provides an interactive UI for document management and real-time chat. It communicates with the backend via REST APIs.
- **Backend (FastAPI):** Handles all business logic, asynchronous task processing, and API routing.

- **Document Parsing (PyMuPDF4LLM):** Converts PDFs into structured Markdown, preserving tables, lists, and hierarchical structures.
- **Vector Database (FAISS):** An in-memory, high-performance similarity search index optimized for dense vectors.
- **Embedding Model (Sentence-Transformers):** Uses the local `all-MiniLM-L6-v2` model to generate 384-dimensional dense vectors from text chunks.
- **LLM Engine (Google Gemini 3.8 Flash):** Generates human-like answers strictly grounded in the retrieved context passages.

1.4 Implementation Details

1.4.1 Document Ingestion Pipeline

When a user uploads a PDF, the system calculates an MD5 hash to prevent duplicate processing. The document is parsed into Markdown using PyMuPDF to preserve grid data and tables. The Markdown is then recursively chunked into overlapping segments (chunk size of 1000 characters, overlap of 200 characters) to maintain contextual continuity.

1.4.2 Embedding and Indexing

The text chunks are passed through a local Sentence-Transformer model. Processing embeddings locally ensures privacy and reduces API costs. The resulting embeddings are indexed using Facebook AI Similarity Search (FAISS) using the `IndexFlatL2` metric, enabling sub-millisecond retrieval times.

1.4.3 Retrieval-Augmented Generation (RAG)

Upon receiving a user query, the system embeds the query using the same local model. FAISS retrieves the top-K most semantically similar chunks. These chunks are injected into a strict prompt template alongside the user's question, instructing the Gemini LLM to synthesize an answer based *only* on the provided context.

1.5 Advantages & Key Features

- **Prevents Hallucinations:** The system forces the LLM to cite its sources, drastically reducing AI hallucinations.
- **Preserves Tabular Data:** By converting PDFs to Markdown, the system understands structured data (like chemical composition tables) that standard parsers miss.
- **Data Privacy:** Embeddings are generated locally, ensuring sensitive text isn't sent to third-party embedding APIs.
- **Scalability:** The FastAPI backend is asynchronous and production-ready.

1.6 Results and Outputs

The final system successfully answers complex queries spanning multiple pages of a document. In testing with technical metallurgy documents, the system accurately retrieved percentage compositions from embedded tables. Every response includes a drop-down section detailing the exact retrieved sources, displaying the origin file name, page number, and similarity score.

Chapter 2

Project Terms & Concepts

2.1 What is RAG (Retrieval-Augmented Generation)?

Retrieval-Augmented Generation (RAG) is an AI framework that improves the quality of Large Language Model (LLM) responses by grounding the model on external sources of knowledge. Instead of relying solely on the data the LLM was originally trained on, a RAG system first *retrieves* relevant information from a custom database (like your uploaded PDFs), and then *augments* the user's prompt with this retrieved data before asking the LLM to *generate* an answer.

2.2 What are Vector Embeddings?

Vector Embeddings are mathematical representations of text. An embedding model reads a sentence or paragraph and converts its meaning into a long array of numbers (a vector). Texts with similar meanings are plotted close together in this mathematical space. This allows computers to understand semantic similarity, enabling searches for "concept" rather than just exact keyword matches.

2.3 What is FAISS?

FAISS (Facebook AI Similarity Search) is an open-source library designed for efficient similarity search and clustering of dense vectors. Once documents are converted into vector embeddings, FAISS organizes them so that when a user asks a question, the system can almost instantly find the closest matching vectors (the most relevant document chunks) even among millions of records.

2.4 What is a Large Language Model (LLM)?

An LLM is an advanced artificial intelligence system trained on vast amounts of text data to understand and generate human language. In the IntelliDoc project, Google's Gemini acts as the LLM. It takes the context retrieved by FAISS and synthesizes a clear, natural-sounding answer for the user.

2.5 What is Chunking?

Chunking is the process of breaking down a large document into smaller, manageable pieces (chunks) before embedding them. If a whole book was embedded as a single vector, the meaning would be too diluted. By chunking text into paragraphs with slight overlaps, the vector database can retrieve the exact paragraph that contains the answer.

2.6 What is FastAPI and Streamlit?

FastAPI: A modern, fast (high-performance) web framework for building APIs with Python based on standard Python type hints. It is used as the backend engine for IntelliDoc.
Streamlit: An open-source Python library that makes it easy to create custom web apps for machine learning and data science. It is used to build the interactive frontend interface where users chat with the AI.